Връзки между обектите

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Статични данни. Асоциация. Dependencies. Ownership. Design guidelines

Какво са статични член данни?

Член-променлива, споделена между всички инстанции на класа.

Декларира се в класа; дефинира се (и, при нужда, инициализира) извън него — с изключение на inline static (C++17+).

Статичните член-данни имат живот, различен от обектите: инициализират се преди main() или при първия използван translation unit (implementation-defined ordering).

```cpp
class MyClass {
  public:
  static int s_count; // декларация
  MyClass();
};


// source MyClass.cpp
int MyClass::s_count = 0; // дефиниция + инициализация


struct Widget {
  inline static int s_count = 0;
  // C++17+: дефиниция в header
  static constexpr int MAX = 256;
  //константа известна на компилатора
};

//constexpr - смята се по време на компилация
```

Проблеми при статичните данни

Редът на инициализация на статични обекти в различни translation units е неопределен.

Това води до бъгове, когато един статичен обект използва друг статичен обект от друг .cpp файл.

```
extern int valueB;
int valueA = valueB + 1; // може да е неинициализиран

// B.cpp
int valueB = 42;
```

Решение

```
int& safeValue() {
    static int value = 42; // lazy + thread-safe
    return value;
}
//Scott Meyers - пример за Singleton
```

Практически съвети

Избягвайте глобални и статични обекти със сложна логика.
Предпочитайте constexpr, inline static или function-local static.
Ако редът има значение — контролирайте го чрез функции.

Какво представляват връзките между обекти?

В обектно-ориентирания дизайн не е важно само какво прави класът, а и:

Как е свързан с други класове?
Кой притежава кого?
Колко силна е зависимостта?

Основни понятия:
Dependency
Association
Ownership
Lifetime management

Dependency

```
class Logger {
public:
    void log(const std::string& msg);
};

class StudentService {
public:
    void enrollStudent(const std::string& name, Logger&
logger) {
        logger.log("Enrolling " + name);
    }
};
```

Dependency = клас A използва клас B временно

Характеристики:
краткотрайна
без ownership
най-слабата форма на връзка

Dependency - добри практики

Добри практики:
Dependency чрез параметри
Dependency чрез абстракции (interfaces)
Минимална видимост (forward declaration)

Лоши практики:
#include навсякъде
Скрити зависимости
Създаване на обекти вътре в метода

```
void foo() {
    Logger l; // ❌ hard dependency
}
```

Детайли

Какво наистина е Dependency?

Dependency означава:
Клас A не може да бъде компилиран / използван / тестван без клас B

Важно:
Dependency НЕ означава ownership
Dependency НЕ означава дълготрайна връзка
Dependency е използване, не притежание

```
//Antipattern
class StudentService {
public:
    void enroll(const std::string& name) {
        Logger logger;  // ❌
        logger.log(name);
    }
};
```

Ownership

Дефиниция

Ownership = кой отговаря за:
създаване, унищожаване, lifetime

std::unique_ptr → един собственик
std::shared_ptr → споделена собственост
raw pointer / reference → НЕ притежава

Пример

```
class School {

std::vector<std::unique_ptr<Student>> students_;

public:
void addStudent(std::unique_ptr<Student> s) {
students_.push_back(std::move(s));
 }

};
```

Smart Pointers

std::unique_ptr → един собственик

std::shared_ptr → споделена собственост

raw pointer / reference → НЕ притежава

Кое кога да изберем?

Пример

Временно използване: Dependency

Дълготрайна връзка без ownership: association

Ясен собственик: unique_ptr

Родител → дете Композиция

Лош дизайн

```
class School {
public:
   void addStudent(const
std::string& name) {
      students_.push_back(new
Student(name)); // ❌
   }
};
```

Проблеми:

1)неясен ownership
2)leak
3)трудно тестване
4)няма RAII

Добър дизайн

```
class School {
    std::vector<std::unique_ptr<Student>> students_;
public:
    void addStudent(std::string name) {
        students_.push_back(
            std::make_unique<Student>(std::move(name))
        );
    }
};
```

Design Guidelines

Добър дизайн означава код, който е лесен за разбиране, поддръжка и разширяване. Ето няколко практични design guidelines, които се доказват в реални проекти:

1) Ясна отговорност (Single Responsibility)
Всеки клас и функция трябва да има една основна причина да се променя. Ако класът „знае твърде много“ – време е за разделяне.

2) Слаба свързаност, силна кохезия
Класовете трябва да са възможно най-независими един от друг, но вътрешно логично свързани. Dependency ≠ Ownership.

3) Изразителни интерфейси
API-то трябва да „казва истината“ за поведението си. Ако функцията може да се провали – трябва да се покаже в типа на функцията (`std::expected`, error codes), не в документацията.

Design Guidelines

Продължение...

4) Грешките трябва да са очевидни.
По-добре компилационна грешка, отколкото runtime изненада. Да се използва `const`, `=delete`, strong typing и RAII.

5) Композицията е за предпочитане пред наследяването
Наследяването създава твърди връзки. Композицията дава гъвкавост и по-лесно тестване.

6) Проектиране за промяна, не за текущия момент
Най-лошият дизайн е този, който „работи идеално... засега“. Помисли как кодът ще изглежда след 2 години.

Пример със School и Student

```
class School {
public:
    std::string name() const { return "High School #1"; }
};

class Student {
    std::string name_;
public:
    explicit Student(std::string name) :
name_(std::move(name)) {}

    void printSchool(const School& school) const {
        std::cout << name_ << " studies at " <<
school.name();
    }
};

//Още по-добър и професионален пример ще разгледаме в
следващите лекции с абстракция.

//Antipattern:

class Student {
    School* school_; // ❌ кой притежава?
};
```

Въпроси?

Благодаря за вниманието!